# Ronic OS

# Product Specification & Benefits Report

A local-first agent workstation for Windows — one desktop shell where coding CLIs, web agents, MCP servers, bot profiles, memory, skills, workflows, and a sandbox wallet share a single workspace, with no server-side dependency for the core loop.

**Version analyzed:** ronicos 2.1.4 **Report date:** 2026-10-30

**Basis:** Full codebase analysis (Rust ~23k LOC, React/TS, cloud control plane)

---

## ★ Core Functions (flagship capabilities — highlighted)

These five capabilities define Ronic OS and deliver the most value to the user. They are verified against the actual codebase implementation.

### CORE 1 Unified Local-First Agent Workstation

**One desktop shell for every AI tool you use.** Ronic OS consolidates coding CLIs, web agents, MCP servers, bot profiles, memory, skills, workflows, and a sandbox wallet into a single Windows desktop application — **with no server-side dependency for the core loop**. Chats, bots, files, and state all live locally under `~/.omniworkspace` (SQLite + encrypted vault). The cloud is only used for licensing, billing, and optional burst compute.

### CORE 2 Multi-Bot Squads + 3-Pillar Computer-Use Engine

**Run 2–4 AI bots simultaneously, each in its own isolated mini-window.** Each bot gets its own WebView2 profile, virtual cursor HUD, and one of three driver pillars: (1) per-bot floating PiP mini-computer windows with synthetic DOM driving confined to that window, (2) **native Windows UIAutomation** control-tree driving with background InvokePattern/ValuePattern and WM_LBUTTONDOWN posting — the user's real cursor is never touched, and (3) **Set-of-Marks vision grounding** — 960×540 window capture fused with UIA rectangles, numbered badges stamped in Rust, exact mark_id → pixel center registry. A shared **Squad Blackboard bus** ( `tokio::sync::broadcast` , capacity 256 + SQLite persistence) lets a producer bot stream structured packets directly into consumer bots — **no OS clipboard, no cross-contamination, sub-millisecond coordination**. A process-wide **emergency halt flag** can stop all computer use instantly. Squads are a **Pro-licensed** feature.

### CORE 3 Encrypted Local Vault + Offline License Verification

**Military-grade local security with zero-knowledge architecture.** All API keys and secrets live in `secrets.vault` encrypted with **AES-256-GCM**; the key is derived from your passphrase with **Argon2id**. `lock_vault` drops the key from memory. Licensing uses **Ed25519-signed 7-day offline leases** verified locally in <0.1ms against a compiled-in public key — editing the database achieves nothing because the signature covers user_id, tier, hardware_hash, and expiry.

### CORE 4 Visual Workflow Engine with Human Approval Gates

**Build multi-step bot automations with checkpoints you control.** Workflows persist in SQLite + YAML, support up to 8 nodes (LLM reasoning, approval gates, wallet execution, browser automation), and run on schedules (daily, every 15m/1h, cron). **Any step containing "approve" automatically becomes a human approval gate** — the workflow pauses until you review and resume. Per-node checkpoint JSON enables pause/resume from any point, with a drag-and-drop node-graph editor.

### CORE 5 Policy-Gated Multi-Rail Wallet & Quant Terminal

**A deterministic guardrail the AI can never bypass.** The built-in wallet enforces hard limits — max per-transaction USD, daily spending cap, human-approval-above threshold, max slippage (basis points), and a **kill switch** that blocks all agent/trade actions instantly. Every spend passes `policy::evaluate` in Rust and settles as auto_signed / pending_approval (fail-closed review queue) / rejected_by_policy / blocked_killswitch — the model has no override path. Four rails are wired (x402 USDC micropayments, Razorpay UPI, Stripe virtual agent cards, Hyperliquid WS quant rail with 5 bps builder rebate), plus per-bot daily limits, operator circuit breakers, and a sandbox faucet. `live_signer` mode signs everything but external network rails stay physically locked in this build.

## Other Functions (complete feature inventory, A–N)

### A. Bots, Profiles, Group Chats & Co-Worker Protocol

- **Bot profiles:** create, duplicate, and manage local AI bot profiles with custom personas, per-bot flags/permissions, and per-harness account mapping (primary/long-context/fallback).
- **Group chats & squads (chat level):** multi-bot group conversations with @-mention resolution, selectable coordination modes, and round-robin dispatch (Pro-gated squads).
- **Co-worker protocol:** outcome-first system prompt, action-signature dedup (3rd identical failing call blocked), tiered pivot ladder on failures, definition-of-done contracts compiled from the user prompt, and EXPERIENCE.md lesson persistence.
- **Model routing & failover:** five provider types — local Ollama, CLI subprocess (incl. WSL), BYOK API, official OAuth (ChatGPT plan via PKCE), isolated webview — with live health probes.
- **Routines:** schedule any thread (daily HH:MM, every 15m/1h, cron).
- **Teach tasks & takeover:** convert conversations into reusable taught tasks; per-bot human takeover toggle.
- **Attachments:** files threaded through dispatch → answer_bot → run_turn → execute_tool with an **absolute-path allowlist** and **sensitive-path denylist**.

### B. Apps, Providers & Tool Detection

- **Environment scan:** auto-detect installed CLIs, IDEs, and web agents (found/missing/started report).
- **One-click linking:** link detected tools into the workspace.
- **Provider management:** add/update/test AI providers; API keys stored in the encrypted vault.
- **Sign in with ChatGPT:** official PKCE OAuth — dynamic client registration, S256 challenge, local 127.0.0.1 callback, ID-token iss/aud/nonce validation, refresh-token rotation; the chatgpt.tokens.use.direct plan scope lets bots reply from the user's ChatGPT plan without an API key. Refresh tokens stored in the OS keychain (Windows Credential Manager).
- **Custom apps:** override executable paths; add arbitrary local apps; start/stop/restart tool daemons (fleet actions).
- **Harness detection:** scan all AI harnesses, open dedicated harness windows, run prompts directly against binaries or URLs.

### C. Memory Vault (Mem0-Style, Local)

- **Built-in Mem0-style vault:** SQLite memory with 256-dim signed hash-embeddings and cosine ranking, [[wikilink]] graph-edge extraction, and an event journal.
- **Multi-engine routing:** scan engines and route memory by scope (global or per-bot); mount/unmount a companion engine; sync across engines. **Freemium gate:** Mem0 + Obsidian free; Zep/Cognee/Letta and graph/code engines are Pro.
- **Obsidian two-way sync:** import/refresh/delete with 5,000-note / 20MB caps — not just export.
- **Vault actions:** search and journal via a dedicated command group.

## D. Skills, Plugins, MCP Mesh & Code Mode

- **18 prebuilt skills ship in the box:** computer-use, agent-browser, code-review, find-skills, grill-me, improve-codebase-architecture, vercel-react-best-practices (~90 rule files), remotion-to-hyperframes, hyperframes-cli, composio, MinerU (PDF/DOCX/PPTX → Markdown/LaTeX), and 7 NVIDIA skills (cuDF, CUDA-Q, cuOpt ×2, NeMo Retriever ×2, Nemotron policy) — each with a governance skill-card.
- **Skill lifecycle:** save, toggle per-profile, remove, load, and propose skills from chat with approve/reject resolution.
- **Plugin system:** save, toggle per-agent or per-desktop, trust, remove (plugin.yaml manifests); scan project-local plugins.
- **MCP servers:** save, toggle, trust, remove, connect directly; marketplace installs (Notion, Linear, Sentry, Slack, GitHub, Supabase, filesystem, memory, Playwright, Razorpay MCPs) register the real server.
- **Universal MCP/ACP mesh:** MCP alias server (port 8765) + ACP JSON-RPC inbox (port 8766) for peer-app tasks; config auto-merging into known clients (ChatGPT/Claude/Gemini/Grok web, Cursor, VS Code, Claude Desktop/Code, Codex, Gemini CLI, Aider, OpenCode, Grokbot, Hermes); per-app share flags; ask_peer ticketing.
- **Code Mode (Cloudflare-style):** compiles the live MCP/ACP surface into a compact ronic.d.ts (with token estimates), then interprets the model's TypeScript block **in-process** — user JS never runs in Node/Bun; a marker denylist blocks process, fetch, eval, imports, and Node builtins.
- **Local MCP bridge:** bearer-token HTTP MCP server on **127.0.0.1:47331** exposing memory/skills/plugins/files tools to embedded webviews and CLI harnesses; auto-registers into Claude Code, Cursor, VS Code, Claude Desktop configs; holds the attachment allowlist + sensitive-path denylist.
- **Opt-in cloudflared tunnel:** auto-downloaded cloudflared fronts the token-gated MCP alias server with one public URL embedding the bridge token (401 without it); last_mcp_call proves a web harness actually connected.
- **Install from URL / git:** install skills, plugins, and git packages (clone a GitHub repo and register its SKILL.md files).

## E. Marketplace

- **Catalog browsing** of skills, plugins, and tools with a review→listed→removed lifecycle.
- **MCP-backed installs:** installing an MCP item registers the real server.
- **Cloud store backend:** Supabase listings/packs/purchases; pack bodies gated to seller-or-paid-buyer.

## F. Computer, Browser & Site Windows

- **3-pillar computer-use subsystem** (~2,650 LOC) — PiP mini-computer windows, native UIAutomation control-tree driving, and Set-of-Marks vision grounding (see Core 2) — plus a process-wide **emergency halt** every driver checks.
- **Mini computer shell:** per-bot shell IPC — boot/layout, app/tab switching (Files/Chrome/Terminal), sandboxed per-bot file explorer and history; the calling webview's label identifies the bot so cross-bot access is impossible.
- **URL inspector:** SSRF-guarded fetch (blocks private hosts) extracting up to 40 interactive DOM nodes; **allowlisted sandbox commands** for safe shell execution.
- **Site windows & sign-in choreography:** Edge-impersonating UA + client hints via WebView2 CDP; Google/Apple OAuth handed off to real Chrome/Edge on the site's own EBWebView profile; popup windows and blank-page recovery.
- **Bot screen windows:** per-bot private browser child-webviews with isolated profiles, workspace-confined shell cwd, and DOM snapshot read-back (up to 40 controls).
- **Page browser & workspace webapps:** embedded browser with navigation/place/hide controls; positioned webapps; launch desktop apps by id/kind/path.

## G. Coding Agents, IDE & Terminal

- **Coding workspace** with a real embedded PTY terminal (xterm.js + portable-pty, PowerShell/bash) — per-session readers stream output; shared between user terminal tabs and bot mini-computer terminals.
- **Git panel:** live status/stage/restore with porcelain parsing, secret-path masking in diffs, and one-click dispatch to installed CLI agents (claude/aider/codex/gemini/opencode) or opening Cursor/VS Code.
- **IDE agent selection** (choose which harness drives the IDE) and kernel ops (DB diagnostics/maintenance, shadow-git pruning, workspace backups).

## H. Bot Fleet, Live Messaging Gateways & Mission Control

- **Fleet dashboard:** control kernel/bot daemons (status with RAM/PID), routines, fleet log; start/stop/restart daemon processes; ghost-runner toggle; per-bot circuit breaker.
- **Live messaging gateways** (~1,600 LOC): real integrations with **Telegram** (Bot API long-polling), **Discord** (Gateway WebSocket), **Slack** (Socket Mode), and **WhatsApp** (QR-pairing WebSocket). Credentials validated against live platform APIs; tokens stored server-side only, never sent to the frontend.
- **Strict remote firewall:** PID-conflict reclaim kills orphaned gateway processes; remote commands honored only when the channel is connected+enabled and the sender matches the allowlisted owner ID; routes @bot mentions and /screen, /approve slash commands.
- **Bi-directional push:** interactive push alerts to your phone (wallet approval gates, sentinel incidents) with inline buttons; callbacks honored only from the allowlisted owner.

## I. Workflows (Extended)

- **Visual node-graph editor** (xyflow) with node kinds: LLM reasoning, human approval gate, wallet execution, browser automation, bot action, conditional/branch.
- **Execution semantics:** approval-gate and wallet-execution nodes pause the run as paused_editable; Resume approves the gate and continues. One-run-at-a-time mutex, per-step checkpoints, wf-progress Tauri events, steps execute through the real bot engine.
- **Checkpoints:** per-node checkpoint output previews and resume-from-checkpoint merging.
- **YAML mirror:** every workflow mirrored to `~/.ronicos/workflows/<id>.workflow.yaml`.
- **Bot-authored workflows:** bots author workflows via the `save_workflow` agent tool.
- **Example shipped:** Sales & Marketing Funnel Automation (lead-capture → lead-qualification → nurture-conversion → retention).

## J. Wallet, Trading & Quant Terminal (Extended)

- **Sandbox ledger** with micro-USD precision ($1.00 = 1,000,000 micro-USD; integer math, no floating-point).
- **Four rails:** x402 USDC micropayments, Razorpay UPI (₹400 cap), Stripe virtual agent cards, Hyperliquid WS quant rail (5 bps builder rebate) — plus a sandbox faucet (3 top-ups/day) and per-bot circuit breakers.
- **Configurable policy gates** (max/tx, daily cap, approval threshold, slippage, kill switch) plus transaction review and sandbox top-ups.
- **Two-layer quant pipeline:** LLM signal → Rust policy signer; quant dispatch (pair, side, size, analyst, thesis) and position closing.
- **Ledger modes:** sandbox vs live_signer — live signs everything but external network rails stay physically locked in this build.
- **Burst Compute:** wallet-gated OpenRouter wholesale proxy (cost + 20%) via the cloud control plane.

## K. Self-Modify, Self-Repair, Sentinels & Kernel Ops

- **4-tier self-repair engine** (~860 LOC): Tier 1 — 500ms anti-hang watchdog + SQLite WAL unlocker; Tier 2 — CDP DOM selector auto-healer with recorded heals; Tier 3 — shadow-git instant checkpoint rollback; Tier 4 — Harness² recursive script/skill auto-patcher with real node --check syntax probes and trailing-comma JSON healing.
- **Autonomous sentinel fleet:** 5 sentinels (Sentry, Supabase, Vercel, GitHub, Middleware) running real local checks every 30s in a background thread with capped audit tails; scan/acknowledge/remediate.
- **Local extensions:** save, toggle, and generate extensions from a natural-language prompt.
- **Repair doctor:** shadow-backup restore (path-name validated), workspace refresh (release stuck turns, WAL checkpoint), and a self-patcher that rewrites known-bad skill scripts.
- **Kernel ops:** DB diagnostics (page/WAL/freelist bytes), DB maintenance (VACUUM/optimize), shadow-git pruning, whole-workspace folder backups.
- **Harness² engine:** prompt complexity scoring, recursive plan records, budgeted runs, a run library with keep/promote/revert into the bot's soul.
- **Bug reports:** preview and submit — **API keys stripped before saving** (Anthropic/OpenAI/Stripe/GitHub keys, bearer tokens, AWS creds, PEM blocks).
- **Reset workspace** with confirmation guard.

### L. Settings, Themes & System

- **Settings** with runtime themes and density; command palette (Ctrl+K), debug overlay (Ctrl+Shift+D), workspace recovery (Ctrl+Shift+R), embedded terminal (Ctrl+`).
- **Native integration:** file/folder pickers, save-to-disk export, snapshot polling, `ronic://` deep links.

### M. Cloud Control Plane (Vercel + Supabase)

- **Dual-gateway billing:** Stripe (USD) + Razorpay (INR/UPI) webhooks, idempotent via a billing-events ledger.
- **Founder Pass:** atomic claim capped at 500 ($149 / ₹11,999), badge numbers 1–500, founder_lifetime tier with 3 device seats, never downgraded by later billing events.
- **PKCE desktop auth** bounce, Burst LLM proxy, ghost-runner stub (501 until the Firecracker cluster ships), and the Tauri updater manifest endpoint.
- **Schema:** profiles, device_activations, licenses, founder_pass_pool, compute_wallets, billing_events, ghost_runners + marketplace tables; RLS everywhere; money mutations only via service-role RPCs.

### N. Security Model (Cross-Cutting)

- **Scoped Tauri capabilities:** main-window IPC only; third-party webviews get zero IPC.
- **Loopback-only MCP bridge** with a fixed tool whitelist; opt-in public tunnel only.
- **Sensitive-path denylist + attachment allowlist** on model-controlled file access.
- **Argon2id + AES-256-GCM vault**; Ed25519 hardware-bound leases; redacted bug reports.

## User Benefits — Why Users Choose Ronic OS

**1. Total Privacy & Data Sovereignty**

Everything runs on your machine. Conversations, bot configs, memory, API keys, and workflows never leave your disk unless you explicitly opt into a cloud feature. Genuinely local-first in a cloud-everything era.

**2. One App Replaces Ten**

Instead of juggling terminals, browser tabs for different agents, MCP managers, note apps, and scripting glue — you get a single coherent shell. Fewer context switches, less setup, more output.

**3. Run Multiple AI Bots in Parallel, Safely**

Isolated cookie partitions and virtual cursors mean bots never interfere with each other or your main session, and the blackboard bus lets them collaborate at machine speed — multi-agent orchestration at the OS level, not in a chat thread.

**4. You Never Lose Control of Spending**

The policy engine is a hard, deterministic Rust gate — not a prompt instruction. Set a daily cap, per-transaction max, slippage limit, or flip the kill switch, and the AI physically cannot exceed it.

**5. Enterprise-Grade Security Without the Enterprise**

Argon2id, AES-256-GCM, Ed25519 offline licenses, hardware-bound device seats, loopback-only bridges, scoped IPC, redacted bug reports — security architecture that would cost a team months, baked in.

**6. Workflows That Pause for Your Approval**

Approval gates mean the bot does the work and waits for your sign-off on anything sensitive. Checkpoints mean you resume exactly where it paused — no lost state, no re-runs.

**7. Instant Access to a Curated Skill Library**

18 production-grade skills ship in the box — computer-use, browser agents, NVIDIA GPU tooling, MinerU document parsing, Vercel React best practices, HyperFrames video. Installed on launch; no hunting, no setup.

**8. Works Offline, Licenses Offline**

The Ed25519 lease model gives paying users 7 days of full access with zero cloud calls on startup — no "phone home to verify you still exist." Dev mode runs unsigned and every local feature keeps working.

**9. Self-Healing Workspace**

Sentinels watch for problems, shadow backups let you roll back, and the repair doctor fixes the workspace automatically. Bug reports strip your API keys before they're saved.

**10. Built for Power Users**

Command palette, embedded terminal, debug overlay, workspace recovery, deep links, native pickers, Obsidian export — every interaction is designed for speed.

**11. Your Bots Live Where You Already Talk**

Native Telegram, Discord, Slack, and WhatsApp gateways mean your bots reply inside the apps you already use — with a strict owner-allowlist firewall so only you can issue commands, and interactive push buttons so you can approve a spend or acknowledge an incident from your phone. Tokens are validated against live APIs and never exposed to the frontend.

### 12. Safe Agent Code Execution, Not Just Chat

Code Mode compiles your live MCP/ACP tool surface into a TypeScript definition and interprets the model's code **in-process** with a denylist (no process, fetch, eval, imports, Node builtins) — agents orchestrate tools with real code, but untrusted JS never touches Node/Bun or your shell.

## Technical Architecture Summary

| Layer | Technology |
|---|---|
| **Desktop shell** | Tauri 2 (WebView2), Windows |
| **Backend** | Rust (~23k LOC, 45 modules), tokio async, SQLite (rusqlite, bundled) |
| **Frontend** | React 19, TypeScript, Vite 7, Tailwind CSS 4, Zustand |
| **UI extras** | xterm.js terminal, @xyflow/react node graphs, react-markdown + GFM + highlight |
| **Crypto** | AES-256-GCM, Argon2id, Ed25519 (dalek), SHA-256, getrandom, OS keychain (keyring) |
| **Desktop control** | webview2-com, windows-rs (Win32/GDI), uiautomation, portable-pty, image+imageproc (SoM badges) |
| **Messaging gateways** | Telegram Bot API, Discord Gateway WS, Slack Socket Mode, WhatsApp QR |
| **Networking** | reqwest (rustls), tokio-tungstenite (rustls), cloudflared (opt-in) |
| **Cloud control plane** | Vercel serverless (zero runtime deps), Supabase (schema + Edge Functions) |
| **Billing** | Stripe (USD) + Razorpay (INR/UPI), idempotent webhooks |
| **Licensing** | Ed25519-signed 7-day offline leases, hardware-bound, compile-time public key |
| **State** | `~/.omniworkspace/state.db` (SQLite, WAL) + `secrets.vault` |
| **Protocols** | `ronic://` deep links, MCP on 127.0.0.1, CDP browser driving |

**Scale of the surface area:** 212 Tauri IPC commands across 27 feature domains (122 in the central command module), 21 application screens, 12 studio components, 45 Rust backend modules, and 18 bundled skills. In-file `#[cfg(test)]` suites validate policy gates, vault round-trips, sentinel break→detect→remediate→verify flows, the attachment allowlist, and Code Mode sandboxing.